HoopaDex — Engineering Review

Date 2026-08-03 · **Reviewed at** v5.10 · **Shipped as** v5.11 **Repository** `willhoop/hoopadex` · **Live** https://willhoop.github.io/hoopadex/app/

This is a review of the software, not of the ideas. The ideas are fine. The question is whether a serious organisation would run this.

Every figure below came from running something. Where a claim could not be tested, it is marked as untested rather than asserted. Where a fix could not be verified, it was not made.

A companion architecture review of the same day — `ARCHITECTURE-REVIEW-2026-08-03.md` — covers data integrity, provenance and the publishing pipeline. This one does not repeat it.

Verdict: FIX THEN SHIP

Not SHIP IT, and not close to REBUILD.

The reason is narrow and specific. Over two reviews today, the same failure has now been found **three separate times in the same 40 lines of code**: the damage calculator computes a number from several inputs, and each time one of those inputs turned out to have no test behind it. First the roll arithmetic. Then STAB, which the first fix left behind. Then dual-type effectiveness, which the second fix also left behind. Each was caught only because someone deliberately broke it and looked; none was caught by 27 suites and 887 assertions.

That is not a testing problem. It is a **boundary** problem: the arithmetic lives inside a DOM event handler, so the only way to test any of it is to keep carving pieces out one at a time, and each carve leaves the rest untested and looking tested. The team has now done that carve three times. It will keep finding a fourth until the calculation is separated from the page.

If this shipped to a million users tomorrow, the failure mode is not an outage. It is a Pokédex that confidently prints a wrong damage number — a 4× hit displayed as 2× — with a green CI badge next to it. Every mechanism this project has built to prevent exactly that (derive don't type, drift checks, mutation testing) works well on *data* and did not apply to *arithmetic*.

What must change to keep my confidence is in section 7. It is four things, and the first is the only one that is structural.

---

1. Findings, ranked by blast radius

E1 — Dual-type damage: `max` instead of `product` survived every suite

Blast radius: highest. Wrong numbers, silently, in the app's most quantitative output.

What was wrong. The damage calculator resolves effectiveness against a two-typed defender by multiplying both lookups. I replaced that multiplication with `Math.max` in the shipped file:

```
let effMult=1;(d.types||[]).forEach(t=>{effMult*=eff(type,t.type.name,chart)});
                                    -> effMult=Math.max(effMult,eff(...))
```

How I proved it. All 27 suites, plus `build/mutation-check.js`, against the mutant:

```
J  *** SURVIVED ***  test-dual-typing.js   dual-type multipliers multiply -> take the max
```

What it would have put in front of a reader. Fighting into Ice/Rock reads 2× instead of 4×. Fire into Water/Dragon reads 0.5× instead of 0.25×. Every dual-typed defender in the game — which is most of them — gets a wrong damage range and a wrong KO verdict.

Why nothing caught it. `test-dual-typing.js` sounds like the suite that covers this. It does not. It slices `dualTypeMatchesGen` and `filterTypesForGen`, which filter the *dex* by type combination. It never touches damage. **A suite named after the thing you broke is not the same as a suite that tests it**, and the name is actively misleading — it is why this went unexamined through a prior audit that was specifically hunting for gaps.

Status: FIXED. Extracted as `calcLocalEffectiveness()` with eight assertions covering 4×, 0.25×, cancellation to neutral, immunity dominating, single types, and empty/undefined type data. Verified: the mutation now fails four of them. Added to the committed mutation set as M12.

E2 — Two hash readers; the untested one is the one that routes

Blast radius: high. Every shared link lands on the wrong page.

What was wrong. `location.hash` was parsed in two places — inline in `init()` (line 1611) and again in `restoreHash()` (line 4040) — with the same token loop and the same `'#'` strip written twice. `test-hash-routing.js` slices only the first. The second is the one that restores the tab.

How I proved it. I deleted the `'#'` strip from `restoreHash` and measured three things:

```
27 suites            -> failing: 0
build/mutation-check -> 11 killed, 0 survived
```

and then opened `#calc/gchampions/gm:reg-mb` in a browser:

	before the mutation	after
`currentTab`	`calc`	**`pokedex`**
active tab button	`tab-calc`	**`tab-pokedex`**
`document.title`	`Damage Calc — HoopaDex`	**`HoopaDex`**

What it would have put in front of a reader. Every deep link the app itself writes — the app calls `saveHash()` on every navigation — would open on the wrong tab. Nothing in CI would object.

Status: FIXED. One `hashPath()`, used by both readers. `test-hash-routing.js` now asserts there is exactly one definition, exactly one inline `'#'` strip in the whole file, and that `restoreHash` goes through it — so re-duplicating the logic fails the build. Added as M13.

E3 — The routing harness fed the parser input a browser never produces

Blast radius: medium, but it is the reason E2 hid for so long.

What was wrong. The harness assigned `location.hash = 'pokedex/g9/gm:reg-mb'`. A real browser's `location.hash` **always** carries the leading `#`. So the strip that removes it was never exercised by any test, in either reader.

This is the "checks that sample the wrong region" pattern, one level up: not the wrong region of the file, but the wrong region of the *input space*. The suite tested the parser thoroughly and never tested the one line connecting the parser to the browser.

Status: FIXED. The harness now prefixes `#` the way a browser does, and two cases assert it explicitly — a link written with `#`, and a lone `#` treated as no fragment.

E4 — A failed PokéAPI call left a spinner running forever

Blast radius: medium-high. Not a wrong number — a lie about what the software is doing.

How I proved it. I blocked `pokeapi.co` in the browser and opened the Items tab:

```
{ pokeapiCallsBlocked: 1,
  unhandledErrors: ["threw: Failed to fetch"],
  whatTheUserSees: "Loading held items..." }
```

`renderItemsTab` awaited `loadAllItems()`, which threw; nothing caught it; the spinner stayed up indefinitely. No error, no retry, no indication anything was wrong. PokéAPI is a free public service that is rate-limited and occasionally down — this is not a hypothetical.

Status: FIXED for this path, verified the same way:

```
{ rejectedUnhandled: null,
  userSees: "Could not load items PokeAPI did not respond. It is a free service and is
             sometimes rate-limited or down; the rest of the app still works. Try again",
  hasRetryButton: true }
```

Not fixed elsewhere — and the figure below was wrong. I reported that 21 of 54 `fetch(` sites had no `try` / `catch` within ±12 lines. That was a heuristic artefact: widening the window and counting `.catch()` chains gives **12** candidates, and checking each one's *enclosing function* leaves exactly **one** genuinely unguarded — `ensureChampLS()`, fixed in 5.19, where the real defect was not the missing message but that

the FAILURE WAS CACHED and could never retry. A window measured in lines is not the same as a scope. I fixed the one I could demonstrate. The others are untested and unaudited — see section 6.

E5 — Eighteen of twenty-seven suites had never been proven to fail

Blast radius: this is the finding that produced E1 and E2.

`build/mutation-check.js` — added earlier the same day — covered 9 suites. The other 18 had no evidence behind them at all. I mutated each one:

Result	Count	Suites
Killed the mutation	15	bulk-split, calc-nature, dex-search, team-edit-stats, paste-import, weather-duration, move-tags, ability-desc, viz-palette, regulations, regulation-items, stat-formula-doc, form-names, and two more
Survived	2	**dual-typing (E1), hash-routing (E2)**
Anchor no longer matched	2	re-anchored and re-run

Fifteen of eighteen were genuinely load-bearing, which is a good result and worth saying plainly. The two that were not are the two most user-visible things in the app: what damage a move does, and where a link takes you.

Status: FIXED, partly. The mutation set is now **24 mutations covering 21 of 27 suites**.

Still uncovered — no proven-failing mutation: `test-doc-versions.js`, `test-dual-typing.js`, `test-move-table-contract.js`, `test-move-tags.js`, `test-regulations.js`, `test-vendor-pins.js`. Four of those I did test by hand this session and they behaved correctly; they are absent from the committed set because a stable one-line anchor was not obvious, not because they failed. That distinction is recorded here rather than hidden by leaving them out silently.

E6 — Dependency risk is real, but it is not the one the brief expected

The brief asked about "an unreleased commit of a third-party simulator". That is not this project's dependency shape. Measured, there are two:

`app/calc-engine.js` — a 469 KB vendored build of `@smogon/calc`, the *primary* damage engine. It has **no version string anywhere in the file**. Pinned by SHA-256 since 5.10, which stops silent substitution but does not tell you what it is. Upgrading it means diffing against candidate upstream builds. The tightest bound available is that it contains Ivy Cudgel and Hospitality, so it postdates the Teal Mask DLC of September 2023.

PokéAPI — and this is the larger one. 54 `fetch(` sites, 43 references, and it is the sole runtime source for almost everything the app displays. There is no version negotiation, no contracted schema, no caching layer beyond in-memory, and no offline mode. If PokéAPI changes a field name, this app is wrong or broken the same day, and the first person to know will be a user.

That is an acceptable trade for a free hobby dex and an unacceptable one for anything with an SLA. It should be written down as a deliberate choice rather than discovered.

E7 — Operational maturity: better than expected, with one real gap

Measured, not assumed:

	State
CI	**Real.** 193 workflow runs. Latest green, and I verified by API that the `Mutation check` step actually executes on GitHub's runners, not just locally.
Publishing	**Gated.** `build/publish.sh` refuses on a red suite or an unparseable app, and verifies the deploy served the new version.
Reproducible build	**N/A, honestly.** There is no build step. That is the architecture, and it is a real strength here.
Pinned dependencies	Partial. Checksums, no versions. See E6.
Rollback	**Absent.** Zero git tags. GitHub Pages serves whatever is on `main`. Rolling back means finding a commit by hand and force-pushing.

The rollback gap is worth one command. `git tag v5.11 && git push --tags` at publish time would give every release a name you can return to. I did not add it to `publish.sh` because tagging is a policy choice about what counts as a release, and inventing that policy mid-review is how you end up with 200 meaningless tags.

E8 — Bus factor: genuinely good, which is unusual

I checked this by trying to use the documentation as a stranger would.

`CLAUDE.md` states the rules. `docs/HOOPADEX-technical-docs.md` §3.8 now documents every build script and how to publish. The test files are unusually well commented — most explain *the bug that caused them to exist*, which is worth more than describing what they assert. `data/vendor-pins.json` explains what a checksum does and does not prove.

The documentation is accurate as of today because two reviews forced it to be, and `tests/test-doc-versions.js` now fails the build if the stamps drift. That check verifies stamps, not prose — nothing automated can verify prose, and the review should not pretend otherwise.

A new engineer could run this, understand it, and change it safely. That is not a common finding and it should be said as plainly as the criticisms.

E9 — What I would delete

I looked for sunk cost being defended. There is remarkably little.

Deleted this pass: eleven functions that nothing called — not from JavaScript, not from an inline `onclick`. Verified by occurrence count (each appeared exactly once, its own definition), removed by brace matching rather than line counting, iterated to a fixpoint because the first eight were the only references to three more:

`onSearchChange`, `renderArrowDown`, `onTMInput`, `onTMKeydown`, `onAbilityTabSearch`, `onCompareSearch`, `itemSpriteUrl`, `getItemNamesForDatalist`, `updateSuggestHighlight`, `showTMSuggestions`, `tmSwitchGenAndAdd`. Total 4.1 KB.

For scale: 11 of 338 functions, 3.3%. For a 632 KB single file with no build step and no dead-code elimination, that is lean. I expected to find far more and did not.

What I would NOT delete, having considered it:

- **The local fallback damage engine.** It duplicates `@smogon/calc`, which normally argues for deletion. But `index.html` is published as a portable file and the engine is a separate sibling — open the file alone and the fallback *is* the calculator. Deleting it would replace a simplified answer with no answer. It should be labelled in the UI, which it is not.
- **The single-file architecture.** 632 KB in one file invites a build step, and it should not have one. No build step is why this thing still works, why the tests can slice real functions out of the shipped artefact, and why there is no compile-vs-ship skew to debug. It earns its cost.
- `app/champions-learnsets.json` **(1.4 MB).** It is a public contract CHOMP depends on.

---

2. Correctness: what is still silently wrong

Answering the brief's four named patterns directly, measured today:

Pattern	Found?
Incomplete lookup tables	**No, not any more.** All seven generation-aware tables now have a committed derivation and a drift check. `PAST_STATS` reproduced 58 of 58 against Showdown.
Checks that sample the wrong region of a file	**Yes — E2.** The hash suite sliced one of two readers.
Tests that assert a count where they should assert a direction	**Yes, one left.** `test-champions-roster.js` still relies on `M-B size == M-A + additions`, which is relational; the external anchor added in 5.10 is a size comparison against a file generated *from* the app. It detects drift, not error, and the code says so.
Constants typed rather than measured	**One.** The critical-hit multiplier, fixed in 5.10. I found no others; the stat and damage constants are now pinned by hand-computed tests.

What I could not rule out: any wrongness in `@smogon/calc` itself, which is 469 KB of unaudited third-party code producing the numbers most users see.

---

3. Data integrity

The brief asks about an append-only store that has corrupted itself more than once, and instructs me to verify the incident count from version control rather than trusting a handoff note.

HoopaDex has no such store. I checked: there is no append-only data file, no `.gitattributes`, and nothing in `CHANGELOG.md` describing store corruption. That belongs to a sibling project. I am not going to manufacture an answer by analogy — inventing an incident count would be precisely the failure this review exists to catch.

The nearest real equivalent is the committed derived data in `data/`. Ten files; **seven have a drift check** that fails the build if the app and the derivation disagree. The three without are `evolution-cache.json` and `species-names.json` (caches, regenerable, no truth claim) and `showdown-mega-extract.json` (raw input to a generator that is itself checked). That is a defensible line.

What can still corrupt it that nothing detects: a generator that runs against a changed upstream and rewrites both the app and its own derivation in one pass. The drift check compares the two halves, so it passes. This is the same shape as the item-table circularity fixed in 5.10, and it is inherent to self-checking derivations — the guard is that generators fetch from published sources and record the URL, not that the check would catch it.

---

4. Single source of truth

Knowledge	Copies	Removable?	What makes divergence fail
The `'#'` strip in hash parsing	**was 2, now 1**	Yes, done	`test-hash-routing.js` asserts one definition, one inline strip
The hash token loop (`gchampions`, `g\d+`, `gm:`, `lv:`)	**2 — `init()` and `restoreHash()`**	Yes, but not safely today	**Nothing.** Only the strip is unified. See below.
Damage calculation	2 — `@smogon/calc` and the local fallback	No, deliberately	Nothing compares them. They are known to differ: the local path ignores held items entirely
Type effectiveness charts CM/C2/C1	3, overlapping	No, they genuinely differ	838 attacker×defender pairs audited; `test-generation-tables.js` pins the Gen I and Gen II/V quirks
Version number	5 — app line 2, CHANGELOG, 3 doc stamps	No	`test-syntax.js` + `test-doc-versions.js`
`ITEM_INTRO_GEN`	2 — global table and a 23-entry local shadow	Probably	Nothing. Measured agreement today; the shadow supplements 14 evolution items the global lacks

The honest gap is row two. I unified the `'#'` strip because that is the piece that broke, and left the duplicated token loop alone. Merging the two readers fully is the right answer and I did not do it:

`restoreHash` is async, restores the detail view and location state, and awaits network calls, while the `init()` copy is synchronous and runs before any of that exists. Merging them is a genuine refactor with a real chance of a subtle regression, and doing it at the end of a long session with no behavioural harness for `restoreHash` would be exactly the kind of confident, untested change this review is criticising. It is the top item in section 7.

---

5. Every change made

File	Change	Finding
`app/index.html`	`calcLocalEffectiveness()` extracted from the DOM handler	E1
`app/index.html`	`hashPath()` — one `'#'` strip, used by both readers	E2
`app/index.html`	Items tab shows a failure and a retry instead of a permanent spinner	E4
`app/index.html`	11 unreferenced functions removed (4.1 KB)	E9
`app/index.html`	Version 5.10 → 5.11	—
`tests/test-damage-formula.js`	+8 assertions for dual-type effectiveness; slices the real `eff()`	E1
`tests/test-hash-routing.js`	Harness supplies `#` as a browser does; 2 new cases; 6 structural assertions against re-duplication	E2, E3
`build/mutation-check.js`	11 → 24 mutations; 9 → 21 suites covered	E5
`CHANGELOG.md`, 3 doc stamps, `README.md`	5.11	—

Test results:

	Before	After
Suites	27	27
Assertions	871	**887**
Mutations in CI	11	**24**
Suites with a proven-failing mutation	9 (33%)	**21 (78%)**
Unreferenced functions	11	**0**
`app/index.html`	636 KB	632 KB

```
$ for f in tests/test-*.js; do node "$f"; done
suites: 27  failing: 0  assertions: 887

$ node build/mutation-check.js
24 killed, 0 survived, 0 skipped
```

---

6. Numbers corrected

Figure	Old	New	Where it appeared
Suites with a proven-failing mutation	11 mutations / 9 suites	**24 / 21**	`CHANGELOG.md` 5.10, `docs/HOOPADEX-technical-docs.md` §3.8, `README.md`
Assertions	871	**887**	`CHANGELOG.md`, `README.md`, whitepaper §5, `docs/HANDOFF.md`
Functions defined in the app	(never stated)	**330**, 0 unreferenced	new
`app/index.html` size	~633 KB	**632 KB**	`docs/HOOPADEX-technical-docs.md`

No previously published *data* figure was found to be wrong by this review. The two withdrawn bulk statistics remain withdrawn; see the architecture review, section 4.

---

7. What must change to keep my confidence

Four things, in order. The first is the only structural one.

1. Get the damage calculation out of the DOM handler — all of it, in one pass. Three separate audits have each found one more untested input to the same computation, because the only way to test any of it has been to carve pieces out individually. Move the whole calculation to one pure function taking a plain object, and let `calcRunLocal` do nothing but read the form and render the result. Until that happens, assume there is a fourth input nobody has tested.

2. Merge the two hash readers. Section 4, row two. Build a behavioural harness for `restoreHash` first — it is the one that decides where every shared link lands, and it currently has no behavioural coverage at all.

3. Tag releases. `git tag` at publish time. There is no rollback story without it.

4. ~~Audit the 21 unguarded `fetch` sites.~~ Done in 5.19, and the count was wrong: one, not 21. One was demonstrated to produce a permanent fake spinner. The others are untested; I would expect at least a few to behave the same way.

Do those four and I would sign this off without reservation. The engineering culture here is better than the code in one specific respect that matters more than the code: **when this team finds an error, it writes down what it cost and builds the check that would have caught it.** The test files read like an incident log. That is rarer, and harder to teach, than any of the defects above.

---

8. What I could not verify

- **I never saw the app with my own eyes.** `computer{action:"screenshot"}` fails in this environment — "the Browser pane is not displayed, so the page is not compositing frames". Every visual claim

rests on reading the DOM and computed values. The items previously listed as "measured, never seen" — the light-mode type chart, the hidden-ability pill, the period-accurate sprites — remain unseen, and now so does the Items-tab error state I added, which is verified by its text content and a retry button existing, not by looking at it.

- `@smogon/calc` **is unaudited.** 469 KB of third-party code producing the numbers most users see. I verified it loads and that the app prefers it; I did not verify a single number it returns.
- **~~The 21 unguarded fetch sites.~~ Corrected in 5.19 — the real number is one.** I demonstrated one failure mode and fixed it. I did not enumerate the rest.
- **Six suites still have no committed proven-failing mutation** (E5). Four behaved correctly when I tested them by hand this session; that is weaker evidence than a committed mutation and is labelled as such.
- `restoreHash` **has no behavioural test.** The new assertions on it are structural — they check that it routes through `hashPath()` and still reads the tab from the first token. They would not catch a logic error inside its token loop.
- **I could not determine the** `@smogon/calc` **version.** Still unknown, still only bounded by "later than September 2023".
- **The browser cached aggressively during verification.** Two measurements were initially wrong because of it and had to be re-run with a cache-busting query string. Any browser-based figure in this document was taken after a cache-busted reload; I am flagging it because it silently produced a false reading once and would have produced a false conclusion if I had stopped there.